Three Roles, One Pipeline

Engineering 101: data, prompt, and context engineering in plain language. These can be three people, one person in three hats, or a team ritual. The map is about who answers for it, not job titles.

	Data engineer	Prompt engineer	Context engineer
Owns	Truth of the inputs: freshness, grain, lineage, access, quality	The ask: task shape, constraints, output form, refusal language	The window: retrieval, permissions, citations, redaction, and when to stop
Question it answers	Is the record true, current, and allowed to be used?	Did we ask for the right artifact, with the right limits?	What may the model see, what may it never invent, and what does a stop look like?
Fails first when missing	Fluent answers on stale or wrong facts	Good data, chaotic asks, noisy output	Right prompt, right warehouse, wrong slice in the window
Gateway	G1 Use	G2 Prompt	G3 Context and G4 Self-check
Monday test	Pick one field the agent reads. Can you name its source, refresh time, and who may see it?	Read your system prompt aloud. Does it say what the agent may never do, and what to output when it cannot answer?	Ask the agent something outside its role. Does it refuse, cite, or hand back, and is that logged?

Contra-examples to watch for

- **Happy-path demo theater:** clean sample data in the demo, missing joins and delayed feeds in production. The prompt never changes; the outcomes do.
- **Prompt cosplay:** a week spent on tone and temperature while nobody owns freshness or grain.
- **Context stuffing:** the whole wiki pasted into the window. Noise crowds out the one paragraph that mattered.
- **Rubber-stamp human:** the UI says human in the loop, but the reviewer has seconds and no refuse button that logs why.
- **No hand-back language:** the model is never taught how to quit, so it always produces something.

After each one, ask: which role failed first, and which of the five stop questions would have caught it?